ABAP OOPS

Foundation of ABAP OOPS

1. What is OOP?

➢ OOP (Object-Oriented Programming) is a way of organizing your program into objects.

Objects = data (attributes) + behavior (methods).

❑ Key concepts:

➢ Encapsulation : Keep data safe by controlling access.

➢ Inheritance : Reuse and extend existing classes.

➢ Polymorphism : One interface, multiple implementations.

➢ Abstraction : Hide details, show only what's needed.

Basic ABAP OOPS Syntax

Step 1 : Class Definition

```
*Step 1 - Class Definition
CLASS lcl_hello DEFINITION.
  PUBLIC SECTION. "Visible to everyone
    DATA: name TYPE string.
    METHODS: say_hello.
ENDCLASS.
```

Step 2 : Class Implementation

```
*Step 2 - Class Implementation
CLASS lcl_hello IMPLEMENTATION.
  METHOD say_hello.
    WRITE: / 'Hello welcome to OOPs course by ',name.
  ENDMETHOD.
ENDCLASS.
```

Step 3 : Create and call object

```
*Step 3 - Create and Call Object
START-OF-SELECTION.
  DATA(lo_obj) = NEW lcl_hello( ). "Object creation
  lo_obj->name = 'Amir'.

  lo_obj->say_hello( ).            "Method call
```

Explanation

Let's go deeper into Code so you don't just "know" ABAP OOP basics, but you own them.

We'll cover:

1. How ABAP OOP works inside SAP (conceptual view)
2. Class structure & memory flow
3. Objects in ABAP references, creation, and destruction
4. Attributes in depth (instance, static, constants)
5. Methods in depth (IMPORTING, EXPORTING, CHANGING, RETURNING)

• How ABAP OOP Works Inside SAP

Think of ABAP OOP as **blueprint** + **object**:

- **Class** : The blueprint (like a plan for building a car)
- **Object** : The actual instance in memory (the real car)

SAP keeps your classes in the Class Builder (SE24) or embedded inside reports/programs.
When you run the program:

- The class definition tells the compiler "what can be done".

```
*Step 1 - Class Definition
CLASS lcl_hello DEFINITION.
  PUBLIC SECTION. "Visible to everyone
    DATA: name TYPE string.
    METHODS: say_hello.
ENDCLASS.
```

Explanation

- The class implementation tells the compiler “how it’s done”.

```
*Step 2 - Class Implementation
CLASS lcl_hello IMPLEMENTATION.
  METHOD say_hello.
    WRITE: / 'Hello welcome to OOPs course by ',name.
  ENDMETHOD.
ENDCLASS.
```

- You create an object in memory using NEW.
- You call methods on the object.

```
*Step 3 - Create and Call Object
START-OF-SELECTION.
  DATA(lo_obj) = NEW lcl_hello( ). "Object creation
  lo_obj->name = 'Amir'.

  lo_obj->say_hello( ).            "Method call
```

Explanation

• Class structure & Memory flow

Every ABAP class has:

```
CLASS class_name DEFINITION.
  PUBLIC SECTION.
    " Methods & Attributes visible everywhere
  PROTECTED SECTION.
    " Visible only to subclasses
  PRIVATE SECTION.
    " Visible only in this class
ENDCLASS.

CLASS class_name IMPLEMENTATION.
  " All method bodies go here
ENDCLASS.
```

Memory flow example:

```
DATA(lo_obj) = NEW lcl_demo( ).
```

- **lo_obj** is a reference variable (it doesn't store data itself, only a pointer to the object).
- NEW creates the object in memory and returns the reference.
- If you set **lo_obj = INITIAL**., the object is garbage collected automatically.

Explanation

• Object in ABAP - References

In ABAP OOP, all objects are handled via references.

```
DATA(lo_obj1) = NEW lcl_demo( ).
DATA(lo_obj2) = lo_obj1. " Both point to the
                           same object
```

If you change data via **lo_obj2**, it also changes for **lo_obj1** because both point to the same object in memory.

```
START-OF-SELECTION.
  DATA(lo_obj1) = NEW lcl_hello( ). "Object creation
  DATA(lo_obj2) = lo_obj1.          "both point to same object
  lo_obj2->name = 'Amir'.

  write:/'LO_OBJ1 ', lo_obj1->name.
  write:/'LO_OBJ2 ', lo_obj2->name.
```

```
LO_OBJ1  Amir
LO_OBJ2  Amir
```

Output

- Here in the above example an **lo_obj1** is instantialize using lcl_hello class
- Then **lo_obj2** is also pointing to the same object in the memory
- When we assign the name using lo_obj2
- The name gets assigned to lo_obj1 as well

Explanation

• Attributes in depth

Attributes are variables inside a class.

a) Instance Attributes (default)

- **Belong to an object, different for each object.**
- `DATA: name TYPE string.` **" Instance**

b) *Static Attributes*

- **Shared across all objects of the class.**
- `CLASS-DATA: count TYPE i.` **" Static**

c) Constants

- **Never change, good for fixed values.**
- `CONSTANTS: pi TYPE p DECIMALS 2 VALUE '3.14'.`

NOTE: We will look in more Depth the use of Attributes in upcoming days

Explanation

• Methods in depth

Methods = functions of a class.

Parameters Types :-

IMPORTING : Input only

EXPORTING : Output only

CHANGING : Input and output (can be modified inside)

RETURNING : Single output value (recommended for simple results)

```
METHODS: calculate
  IMPORTING first TYPE i second TYPE i
  RETURNING VALUE(result) TYPE i.
```

NOTE: We will look in more Depth the use of all the parameters in upcoming days

Basic ABAP OOPS

❑ **Key Points:**

- **PUBLIC SECTION :** accessible anywhere
- **PRIVATE SECTION :** accessible only inside the same class
- **PROTECTED SECTION :** accessible inside class, subclass & friend class
- **METHODS keyword :** declares functions inside a class
- **NEW :** creates an object

Exercise

Task:

1. Create a class lcl_math
2. Add a method add_numbers that takes two integers and prints their sum.
3. Create the object and call the method.
4. Hints:
5. Use IMPORTING parameters for input
6. Use WRITE to show result

Challenge

Challenge Make It Dynamic :

After you do the above Excercise, try:

1. Adding a subtract_numbers method
2. Using RETURNING parameter instead of just WRITE